

Land Deal CRM — Product and Engineering Plan

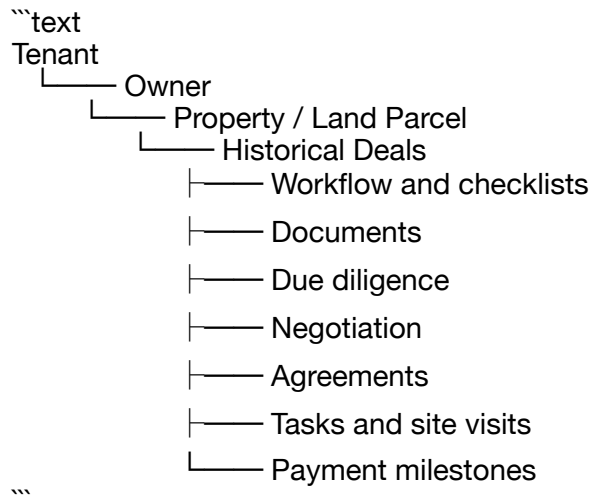
Decision

Proceed with building a multi-tenant Land Deal CRM centered on the property/land parcel rather than a conventional contact-based CRM.

The system will be built incrementally as a cloud-neutral modular monolith with independently deployable web, mobile, API, and worker applications.

Product vision

The core business relationship is:



The default acquisition journey is:



The workflow must be configurable, auditable, and capable of supporting different land types, states, and tenant requirements.

Confirmed product decisions

Users and scope

- Version one serves internal acquisition teams.
- Initial users include acquisition, management, legal/technical reviewers, and finance-oriented users.
- Broker portals and external access are future capabilities.
- The product is multi-tenant SaaS.
- India is the initial market, with configurable state, district, tehsil, and village terminology.
- Legal rules must not be hardcoded for one state.

Domain model

- `Owner`, `Property`, and `Deal` are separate entities.
- One owner can have multiple properties.
- One property can have multiple historical deals.
- One active acquisition deal is allowed per property by default.
- Authorized administrators can override the active-deal restriction for exceptional cases.
- Ownership is stored on the property-owner relationship.
- Each deal stores a snapshot of ownership when the deal begins or ownership is confirmed.

Deal management

Every deal has:

- Configurable workflow stages
- Stage history
- Stage-specific checklists
- Tasks and due dates
- Assigned users
- Due-diligence status
- Negotiation and agreement information
- Payment milestones
- Audit history

Stage rules:

- Deals may move backward only with a mandatory reason.
- Stages may be skipped only by authorized users.
- Skipped stages and checklist items are explicitly recorded as bypassed, never silently completed.
- Workflow templates are centrally published by system administrators.
- Tenant administrators may activate and configure permitted templates.
- Existing deals retain the workflow-template version they started with.

Documents

The document vault supports ownership, property, and transaction documentation, including:

- Sale deeds and previous sale deeds

- Title, mutation, revenue, khasra, and khatauni records
- Encumbrance and property-tax records
- Legal-heir documents
- Site, survey, demarcation, land-use, conversion, and NOC documents
- LOI, MOU, agreements, registration documents, and payment receipts

Every document has:

- Classification
- Tenant/property/deal scope
- Access policy
- Version history
- Review status
- Uploader and reviewer information
- Audit events
- Optional expiry or renewal date

Sensitive classes include owner identity documents, bank documents, financial documents, and confidential internal documents. These require explicit permissions and complete auditability.

The property and deal views show document completeness, for example `17/24 received`, including missing-document details.

Financial tracking

Version one is not an accounting replacement. It tracks acquisition financial milestones with auditable history:

- Asking price
- Expected purchase price
- Negotiated price
- Advance amount
- Balance amount
- Planned payment milestones
- Actual payments
- Due dates
- Payment status
- Receipts and supporting documents
- Approval and audit history

Money must be stored using safe decimal or minor-unit representations, never binary floating-point arithmetic.

Property maps and site maps

The first map release supports:

- Property map pins
- Status-colored markers
- Latitude and longitude
- Optional GeoJSON parcel boundaries
- GeoJSON and KML uploads
- PDF and image uploads

- Manual tracing for scanned maps
- Area and perimeter calculation
- Nearby roads, cities, and highways

Map processing flow:

```text

Upload original map

- Store immutable original
- Create processing job
- Extract or manually create geometry
- Georeference and validate
- Store versioned GeoJSON in PostGIS
- Technical/legal review
- Approve as active geometry

```

Rules:

- Original uploads are immutable.
- Every derived geometry is versioned.
- A new upload never silently replaces the active geometry.
- The previous approved geometry remains active until the new version is approved.
- Acquisition users can create drafts.
- A technical or legal reviewer approves geometry.
- Each geometry records creator, reviewer, source, status, and history.
- Satellite imagery, terrain, land-use analysis, and advanced site intelligence are future modules.
- Map-derived information is informational until a qualified reviewer approves it; it cannot by itself prove legal title.

The map provider is abstracted. Initial map technology is OpenStreetMap-compatible data with MapLibre GL. Public OSM services must not be assumed to provide unlimited production geocoding, tiles, or routing; those providers remain configurable.

Site visits and scheduling

Site visits are separate entities linked to a property and/or deal. They include:

- Schedule and status
- Attendees
- Assigned users
- Notes
- GPS/location data
- Photos and videos
- Follow-up tasks
- Audit history

Version one supports internal scheduling. Calendar integrations are deferred behind an integration boundary for future Google Calendar and Outlook synchronization.

Site and plot registry

The CRM will include a site/plot registry for inventory derived from uploaded layout maps and property documents. A registry record may represent a land plot, apartment, studio flat, or commercial unit within a larger site or project.

Site identity and change control:

- Site IDs are automatically generated, unique, human-readable, and immutable.
- Initial format: `SITE-000001`.
- The generated ID is prefilled and user-confirmable before creation.
- The system may suggest whether an entry matches an existing site using map/document, plot label, location, and dimensions.
- The user confirms whether the record is new or an update; the system must not silently merge records.
- Every edit preserves an audit history and keeps the latest approved record active.
- New or edited records are saved as pending.
- An administrator can approve or reject the change.
- Normal inventory search exposes approved records by default.

Map/document traceability:

- Store the source map or document ID.
- Store page number where relevant.
- Store an optional map label, region reference, or layout reference.
- Store structured location data plus the original map reference.

Property-type templates:

- Land/plot
- Apartment
- Studio flat
- Commercial unit

Land/plot fields include:

- Area
- Length and width
- Location
- Plot label
- Cut dimensions
- Frontage
- Road and access details
- Notes

Irregular plots store named edges or boundary segments with dimensions and notes, for example frontage, north edge, south cut, and east boundary. A polygon is optional and remains part of the separate reviewed site-map geometry workflow.

Apartment and studio-flat fields include:

- Total area
- Carpet area
- Built-up or super-built-up area
- Rooms
- Bathrooms

- Floor
- Unit number
- Notes

Commercial-unit fields include:

- Total area
- Carpet area
- Built-up area
- Unit number
- Floor
- Frontage
- Access
- Usage type
- Notes

Measurement policy:

- Store the value exactly as entered together with its unit.
- Do not silently normalize or convert the original value.
- Any calculated value must be explicitly marked as derived and retain its source values and formula.
- Structured fields are used for deterministic filtering and validation; notes handle unusual site-specific information.

Inventory search and filtering include:

- Site ID
- Property type
- Map/document
- Plot label
- Location
- Dimensions
- Area ranges
- Rooms
- Floor
- Status
- Approval state

No LLM or Gemma dependency is included in the inventory workflow. Official inventory records are entered, confirmed, and approved by users. Future document extraction can be considered separately, but it must not become an authority for creating or approving inventory.

Scoring and dashboard

The system will support factual deal scoring using configurable factors such as:

- Location and highway proximity
- Land area
- Asking and target price
- Title status
- Road access
- Land use
- Documentation completeness

- Owner readiness

The Deal Control Center should show:

- Property summary
- Deal stage and progress
- Owners and verification status
- Document completeness
- Legal, technical, revenue, and survey statuses
- Asking, negotiated, advance, and balance amounts
- Next action, assignee, and due date

Technical architecture

Recommended stack

```text

Web: React + TypeScript + Vite  
Mobile: React Native + Expo  
API: FastAPI + REST/OpenAPI  
Database: PostgreSQL + PostGIS  
Files: S3-compatible private object storage  
Workers: Redis-backed background jobs  
Maps: MapLibre GL + OpenStreetMap-compatible providers  
Auth: OIDC

```

PostgreSQL is the system of record because it is free/open source and provides strong transactions, relational constraints, reporting, JSONB for configurable fields, full-text search, and PostGIS. MySQL/MariaDB are possible alternatives but are less suitable for this combination of relational workflow and geospatial requirements. SQLite is suitable for local prototypes, not the production multi-tenant database.

Monolith versus microservices

Start with a modular monolith, not microservices.

The API remains one deployable FastAPI application with strict internal modules:

- Identity and tenancy
- Users, roles, and permissions
- Owners
- Properties and geography
- Deals and workflows
- Documents
- Due diligence
- Tasks and site visits
- Payments
- Reporting
- Audit

This is appropriate for hundreds to thousands of users, tens of thousands of properties/deals, and substantial document volume. It avoids premature distributed transactions, duplicated authorization, operational overhead, and difficult cross-service workflows.

The design must make later extraction possible. Document processing, notifications, reporting, search, or integrations can become independent services if they develop independent scaling or deployment requirements.

Separate deployments

Frontend and backend deployments are intentionally separate:

```
```text
Web deployment → FastAPI API
Mobile application → FastAPI API
Worker deployment → PostgreSQL, object storage, queue
```
```

The frontend does not depend on backend implementation details. REST/OpenAPI is the initial boundary, with generated TypeScript clients for web and mobile. GraphQL is deferred unless cross-domain UI queries later justify it.

Suggested repository layout:

```
```text
apps/
 web/
 mobile/
 api/
 worker/

packages/
 api-client/
 domain/
 ui/
 config/

infra/
 docker/
 terraform/
 migrations/
```
```

Frontend strategy

Use React + TypeScript + Vite for the authenticated CRM web application. It is cost-efficient because it builds to static assets that can be hosted cheaply on object storage/CDNs and does not require a frontend server.

Use React Native + Expo for the mobile application. The mobile app will be a real Android/iOS application distributed through the Google Play Store, Apple App Store, or internal enterprise distribution.

Do not force identical UI code across web and mobile. Share API clients, domain types, validation, permissions, design tokens, and selected primitives while allowing each platform to use appropriate interaction patterns.

Next.js remains an optional future application for public, SEO-sensitive pages or portals. It is not required for the core authenticated CRM.

Authentication and authorization

Use OIDC for authentication and keep business authorization in FastAPI.

Confirmed provider strategy:

- Development: self-hosted Keycloak locally with Docker.
- Early production: managed OIDC provider with a free tier, such as Zitadel or Auth0.
- Enterprise expansion: customer-managed Microsoft, Google, or other OIDC providers.

The provider is hidden behind a provider-neutral interface.

Login flow:

```
```text
Web/mobile
→ OIDC provider
→ Authorization code with PKCE
→ Access and refresh tokens
→ FastAPI validates access token
→ Backend resolves tenant, roles, and permissions
```
```

Security rules:

- Web sessions should use secure HTTP-only, same-site cookies or a backend-for-frontend session.
- Do not store long-lived tokens in `localStorage`.
- Mobile tokens use iOS Keychain/Android Keystore through Expo SecureStore.
- Access tokens are short-lived.
- Logout clears local session and cached data, revokes refresh tokens where supported, and logs out at the OIDC provider.
- FastAPI never trusts frontend role claims without validating the token and resolving backend permissions.
- Sensitive operations may require recent authentication or MFA later.

Authorization uses:

- Tenant-level role-based access control
- Property-level assignment
- Deal-level assignment
- Explicit sensitive-document permissions
- Full audit history

Every tenant-owned record carries a tenant boundary. Start with service-layer enforcement and tests; introduce PostgreSQL row-level security before onboarding external tenants at scale.

Cloud and deployment strategy

Keep the platform cloud-neutral with containers and standard interfaces:

- Docker-compatible application containers
- PostgreSQL/PostGIS
- S3-compatible object storage
- Redis-compatible queue/cache
- OIDC-compatible identity provider
- Provider adapters for maps, email, and notifications

The system can run on AWS, Azure, or GCP with limited application changes.

Development can run at no infrastructure cost with Docker Compose:

- PostgreSQL/PostGIS locally
- MinIO for S3-compatible object storage
- Redis locally
- Keycloak locally
- FastAPI, worker, web, and mobile development environments

Cloud free tiers and introductory credits are suitable for prototypes, but production will eventually incur costs for compute, database, backups, storage, monitoring, email, and map services. Free-tier terms must be rechecked before deployment.

Delivery roadmap

Milestone 1 — Platform foundation

- Repository and package structure
- Docker Compose development environment
- FastAPI application shell
- PostgreSQL/PostGIS migrations
- Keycloak local development setup
- OIDC token validation
- Tenant, user, role, and permission foundations
- Audit-event framework
- CI checks and environment configuration

Milestone 2 — Owners and properties

- Owner records and contacts
- Property records and locations
- Property-owner relationships and percentages
- Ownership validation and history
- Property search and filtering
- Initial map pins

Milestone 3 — Deals and workflows

- Historical deals per property
- Active-deal rule and administrative override
- Configurable workflow templates
- Versioned stages and checklists
- Stage transition history
- Backward movement reasons

- Authorized stage skipping
- Tasks, assignments, priorities, and due dates

Milestone 4 — Document vault

- Private object storage
- Upload/download authorization
- Document classifications
- Document versions
- Review status and reviewers
- Missing-document checklist
- Expiry reminders
- Audit events

Milestone 5 — Site-map processing

- GeoJSON/KML upload
- PDF/image upload
- Immutable originals
- Background processing jobs
- Manual geometry tracing
- Geometry versioning
- PostGIS area and perimeter calculations
- Nearby roads, cities, and highways
- Reviewer approval workflow

Milestone 6 — Finance and site visits

- Payment milestones
- Planned and actual payments
- Receipts
- Deal financial summary
- Internal site-visit scheduling
- Site-visit notes, GPS, photos, videos, and follow-up tasks

Milestone 7 — Control center and map dashboard

- Deal Control Center
- Pipeline views
- Property map with status colors
- Deal scoring
- Site/plot inventory search and approval status
- Management dashboards
- Document and due-diligence completeness indicators

Milestone 8 — Production hardening

- PostgreSQL row-level security
- Backup and restore testing
- Monitoring and alerting
- Rate limiting
- Security review
- Offline mobile synchronization
- Push notifications

- Calendar integration boundary
- Scalable document processing

Explicitly deferred

These are intentionally designed for later expansion but are not required for the first build:

- Broker portals
- External user access
- Enterprise per-tenant SSO
- Google Calendar and Outlook synchronization
- OCR and document intelligence
- E-signatures
- Accounting-system integration
- Satellite imagery analysis
- Automated land-use classification
- Terrain and infrastructure intelligence
- Full workflow visual builder
- GraphQL
- Microservices
- CAD/DWG processing

Build principles

1. The property is the center of the product.
2. A deal is an acquisition attempt, not the property itself.
3. Preserve history rather than overwriting important state.
4. Configuration belongs in versioned data where workflows may change.
5. Legal and financial facts require auditability.
6. Derived geographic intelligence must show its source and confidence.
7. Human review remains authoritative for title, ownership, and legal verification.
8. Keep cloud and identity providers replaceable through adapters.
9. Build a modular monolith before extracting services.
10. Make mobile field work a first-class experience, not a shrunken desktop interface.

Go-ahead

The product direction and architecture are sufficiently defined to begin implementation with Milestone 1: platform foundation.